Automated Protocol Analysis using Tamarin

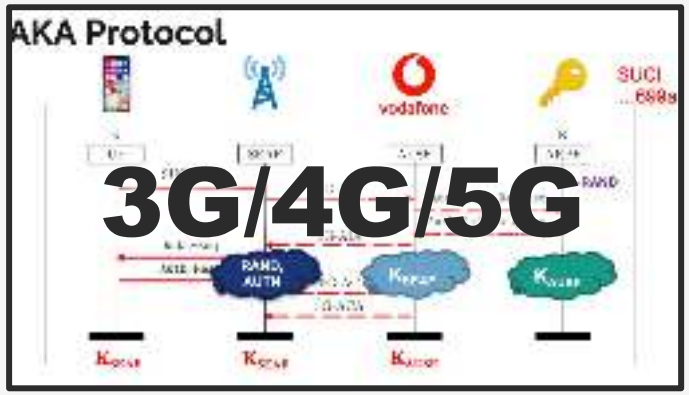
David Basin — ETH zürich

Cas Cremers — CISPA Helmholtz Center for Information Security

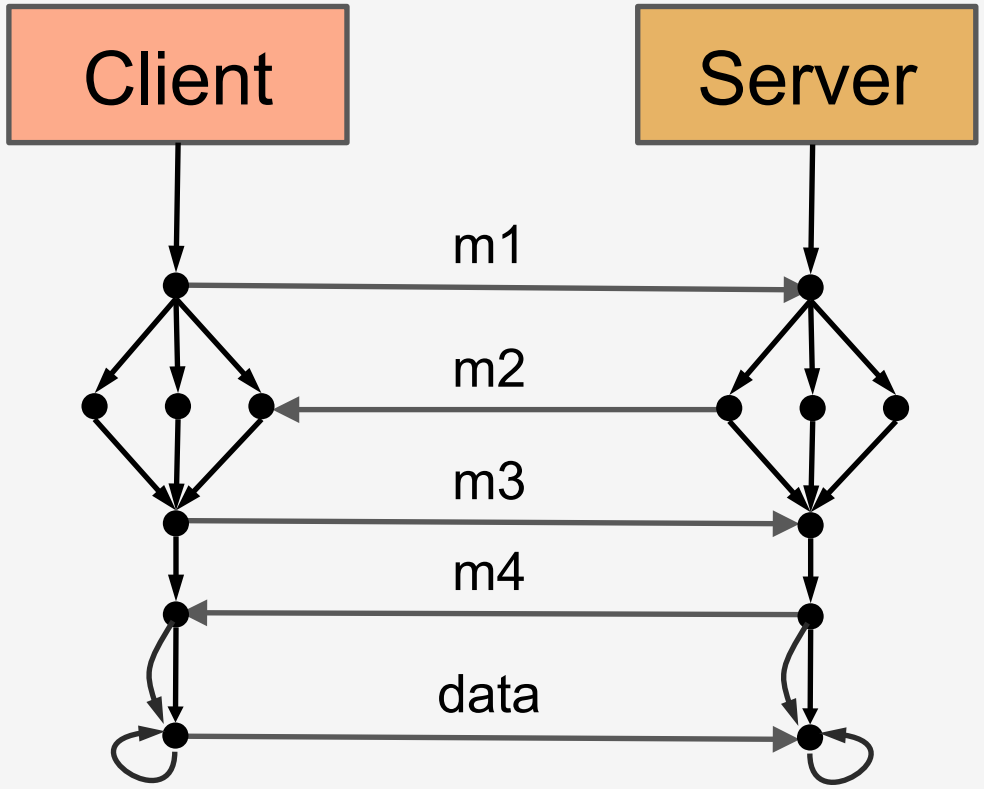
Jannik Dreier — Université de Lorraine, Loria

Ralf Sasse — ETH zürich

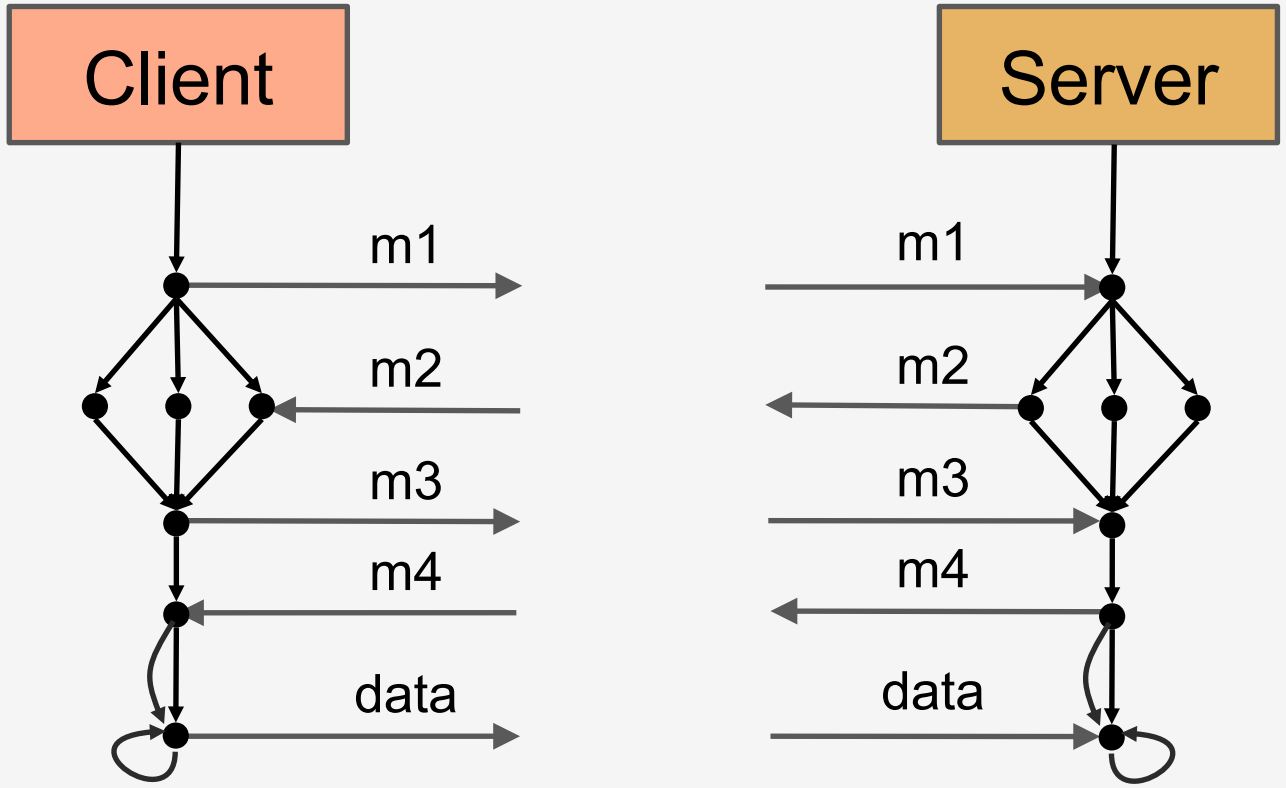
We use security protocols daily



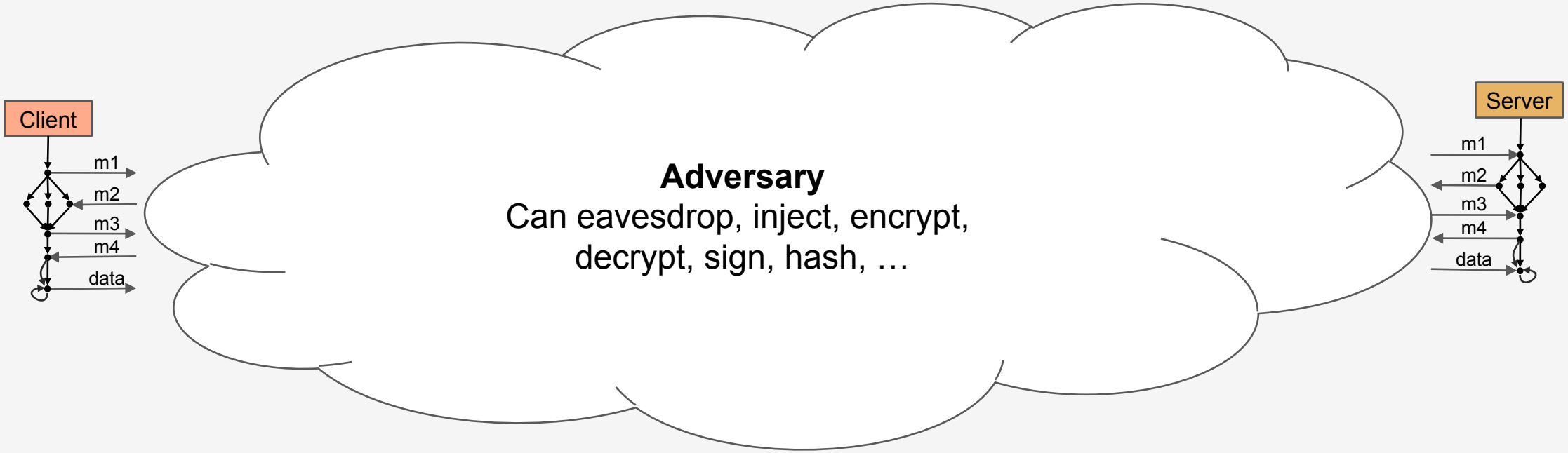
Protocols and threat models



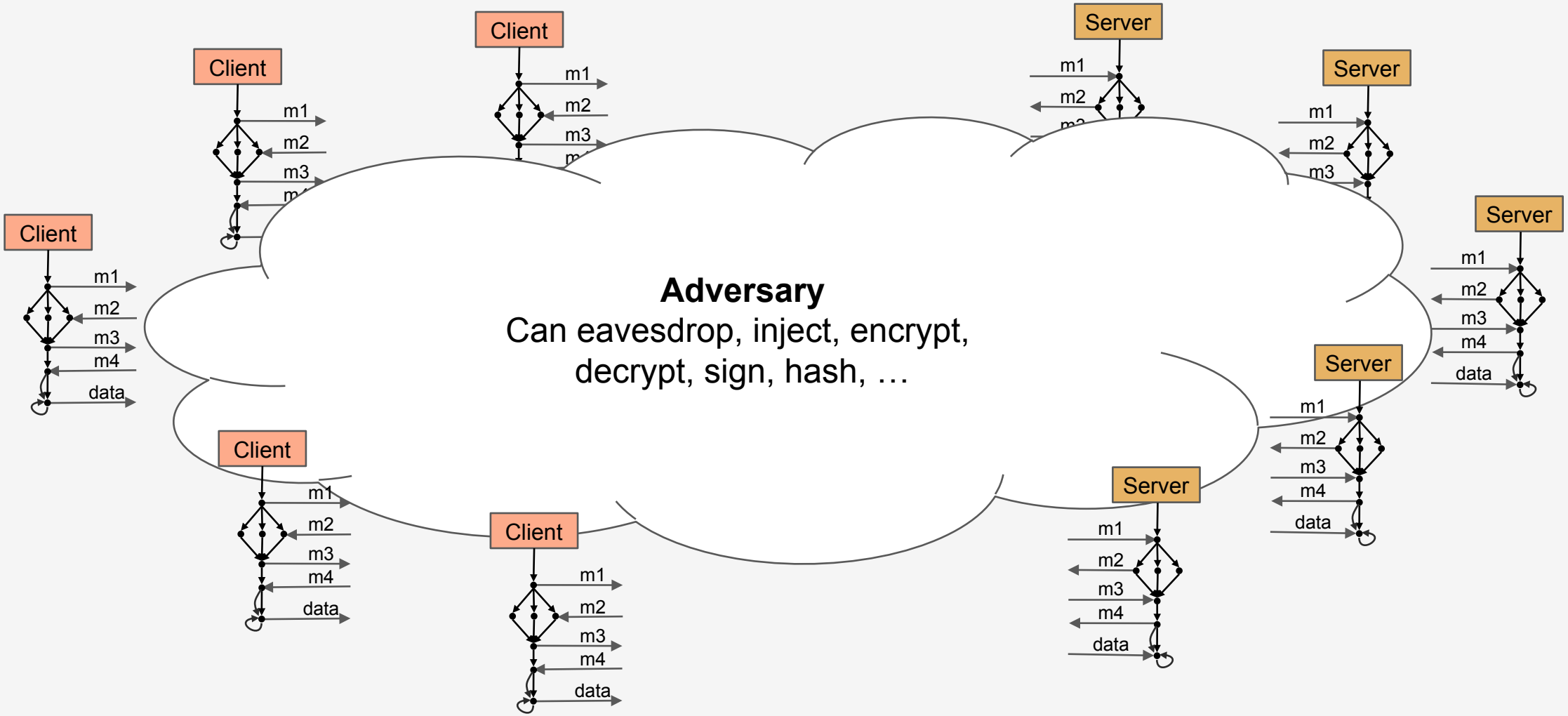
Protocols and threat models



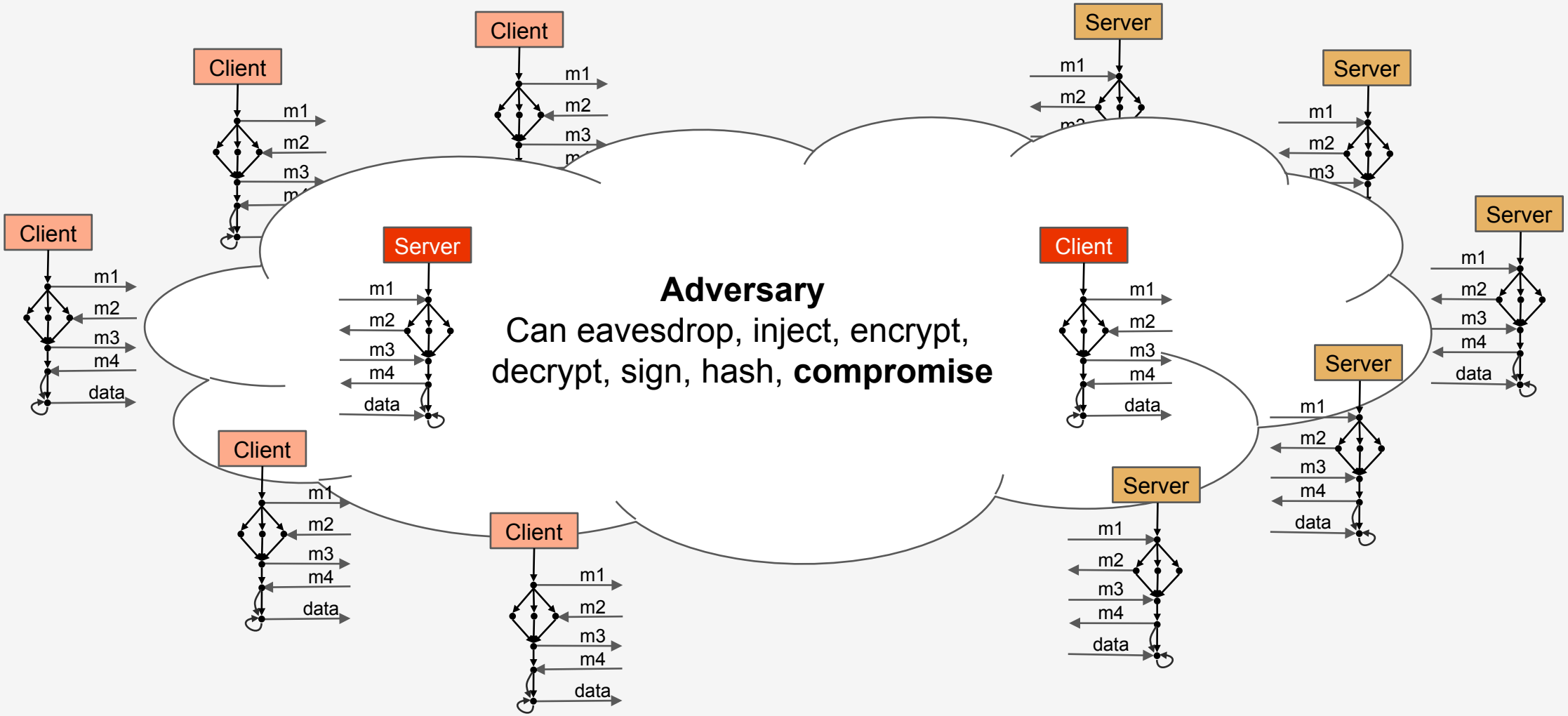
Protocols and threat models



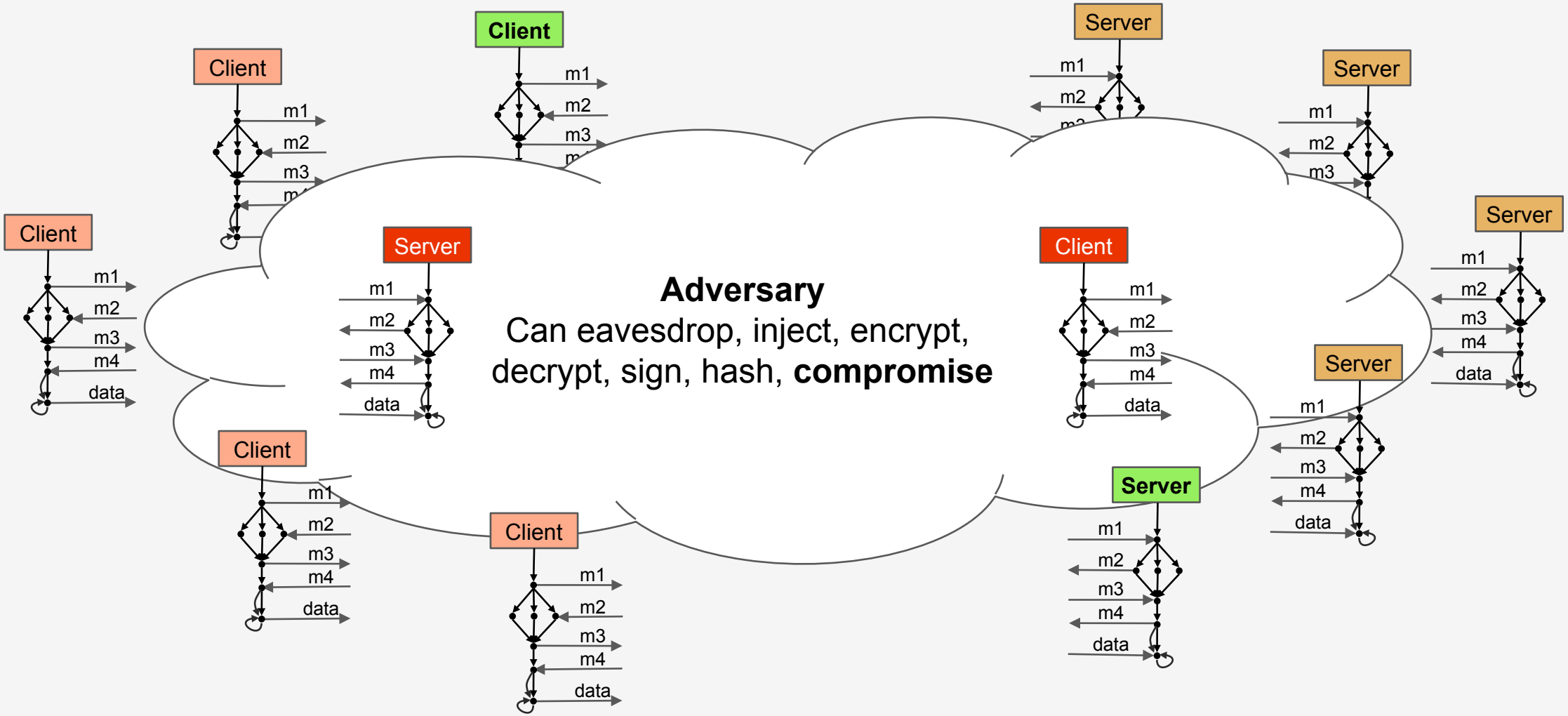
Protocols and threat models



Protocols and threat models



Protocols and threat models



Tamarin prover

Theorem Prover

Constraint solver

Tamarin prover

Emperor Tamarin

Tamarin prover

Emperor Tamarin

Tamarin prover

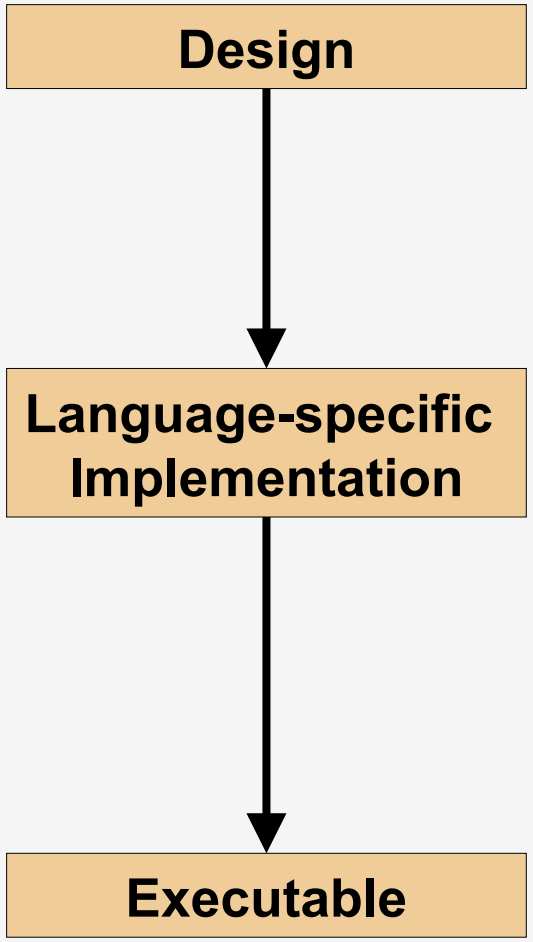
Development core:

- CISPA (Germany: Cremers)
- ETH Zurich (Switzerland: Basin, Sasse)
- LORIA (France: Dreier)

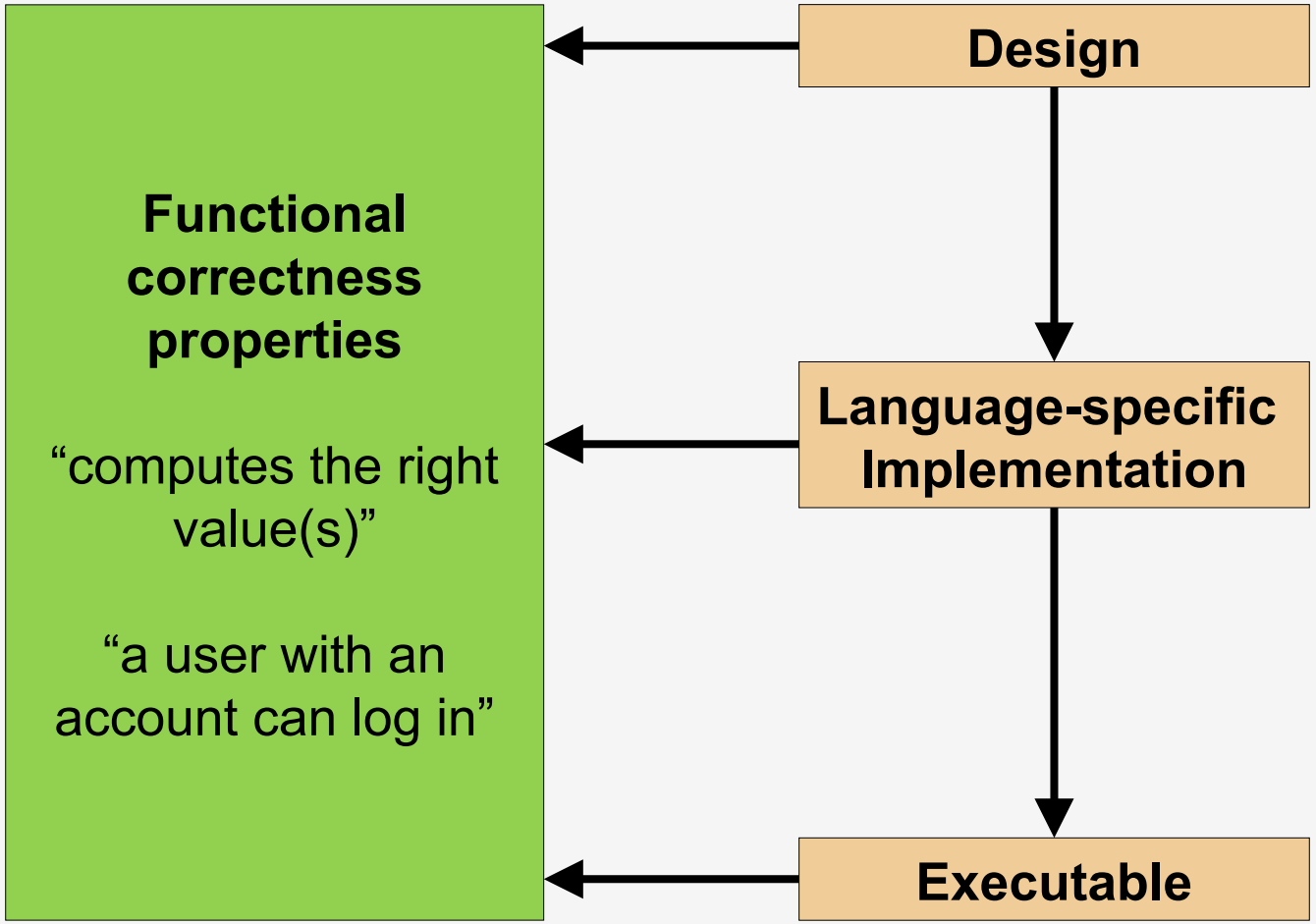
Open-source development

`tamarin-prover.com`

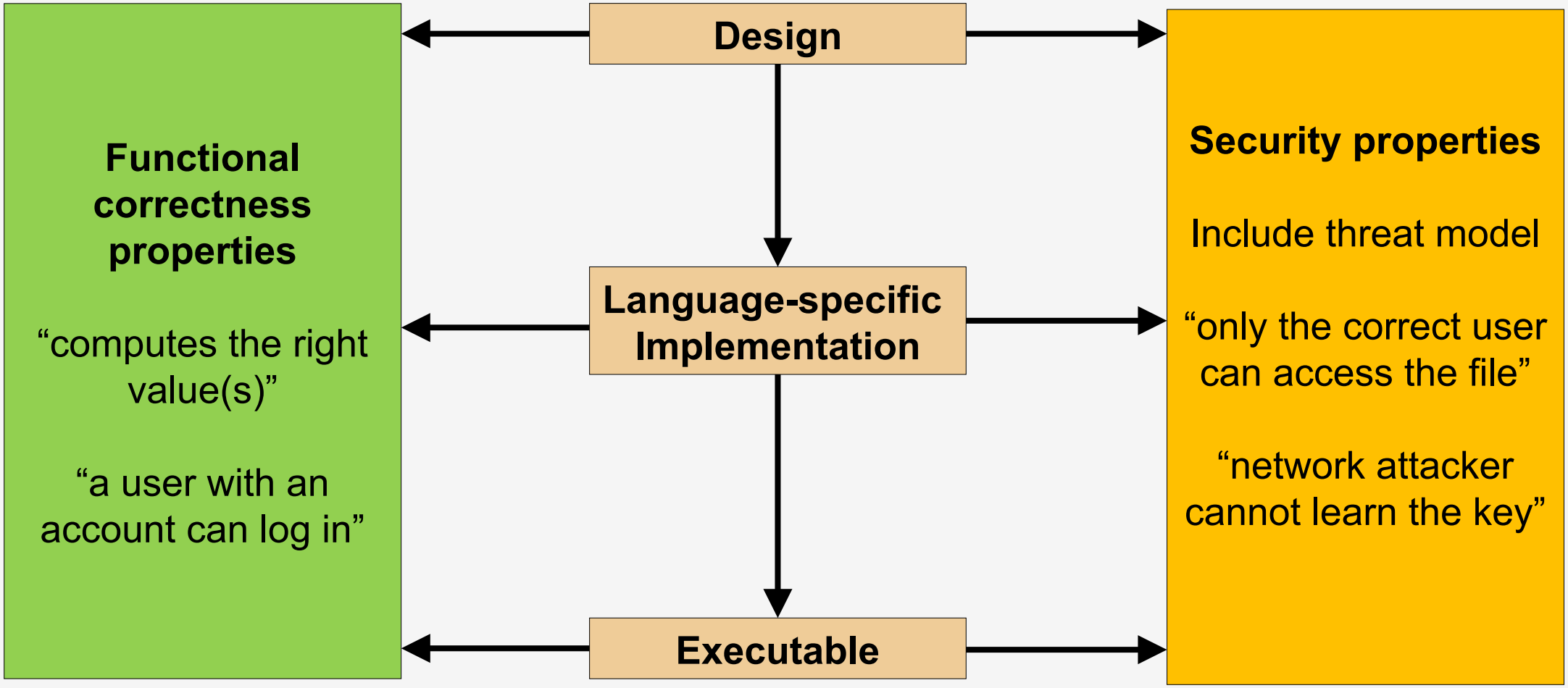
Application domains



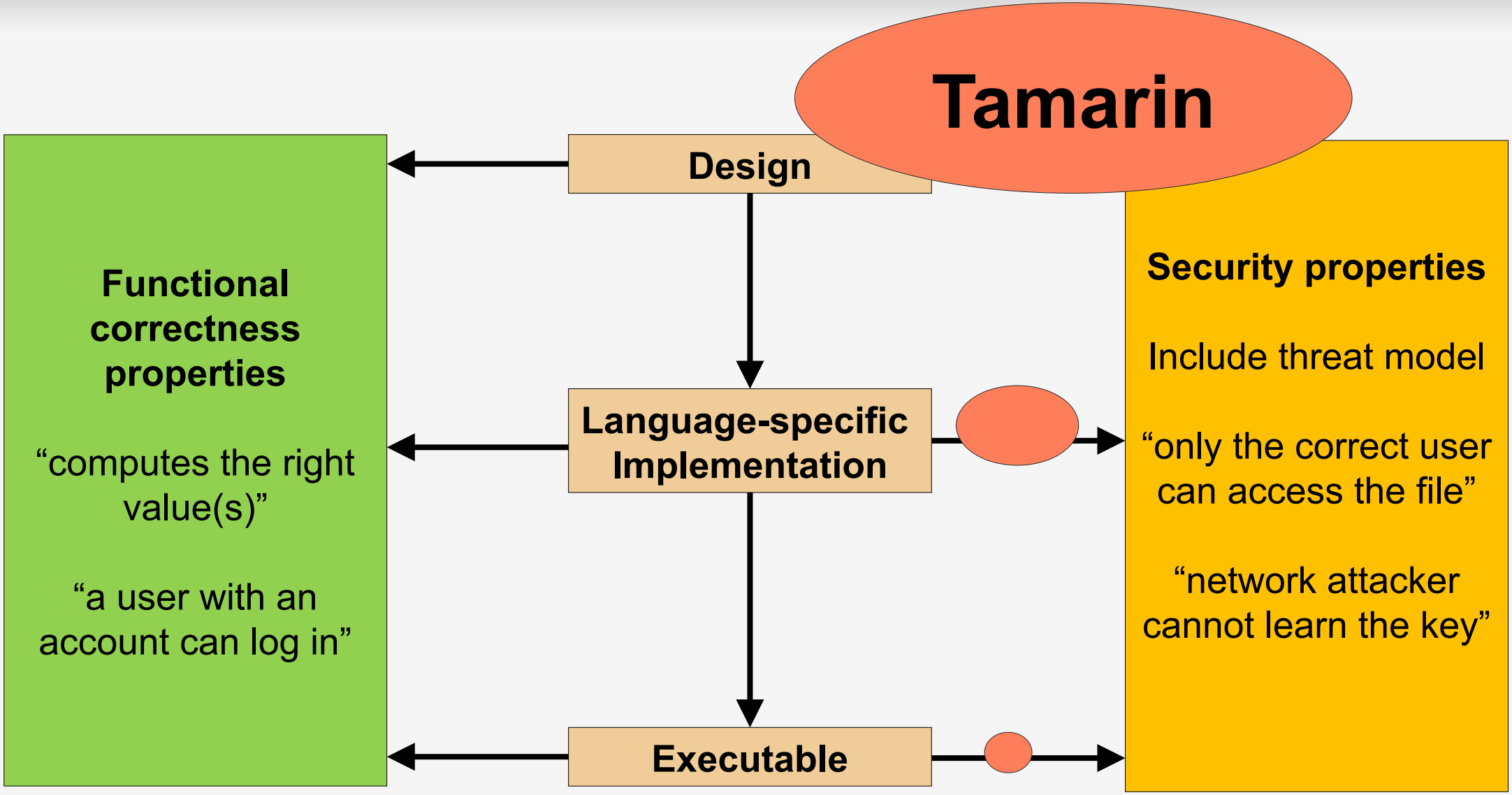
Application domains



Application domains



Application domains



Tamarin prover: workflow

Internet Engineering Task Force (IETF) E. Rescorla
Request for Comments: 8446 Mozilla
Obsoletes: 5077, 5246, 6961 August 2018
Updates: 5705, 6066
Category: Standards Track
ISSN: 2070-1721

The Transport Layer Security (TLS) Protocol Version 1.3

Abstract

This document specifies version 1.3 of the Transport Layer Security (TLS) protocol. TLS allows client/server applications to communicate over the Internet in a way that is designed to prevent eavesdropping, tampering, and message forgery.

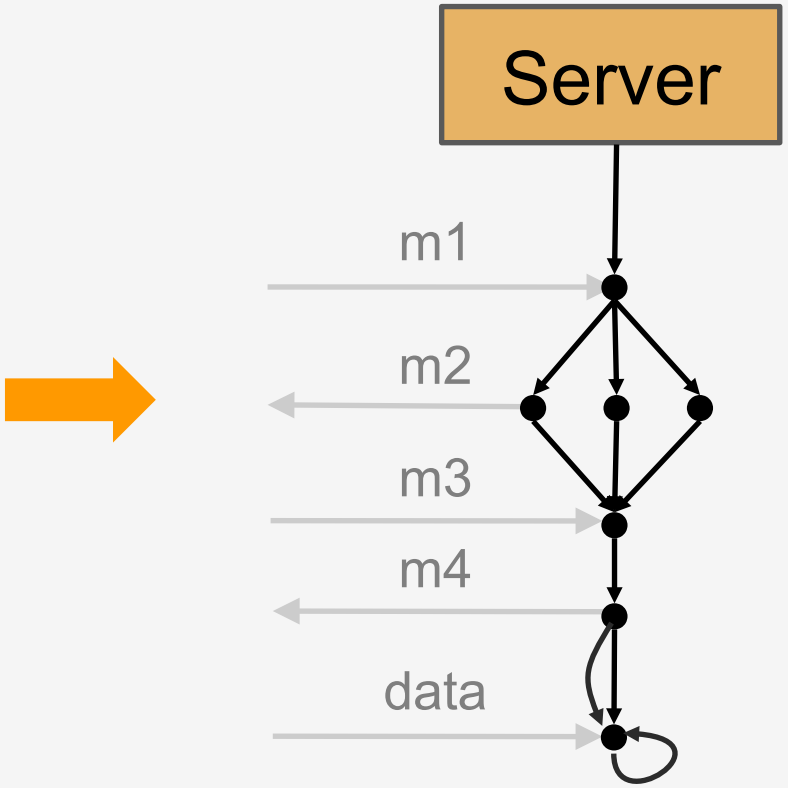
This document updates RFCs 5705 and 6066, and obsoletes RFCs 5077, 5246, and 6961. This document also specifies new requirements for TLS 1.2 implementations.

Status of This Memo

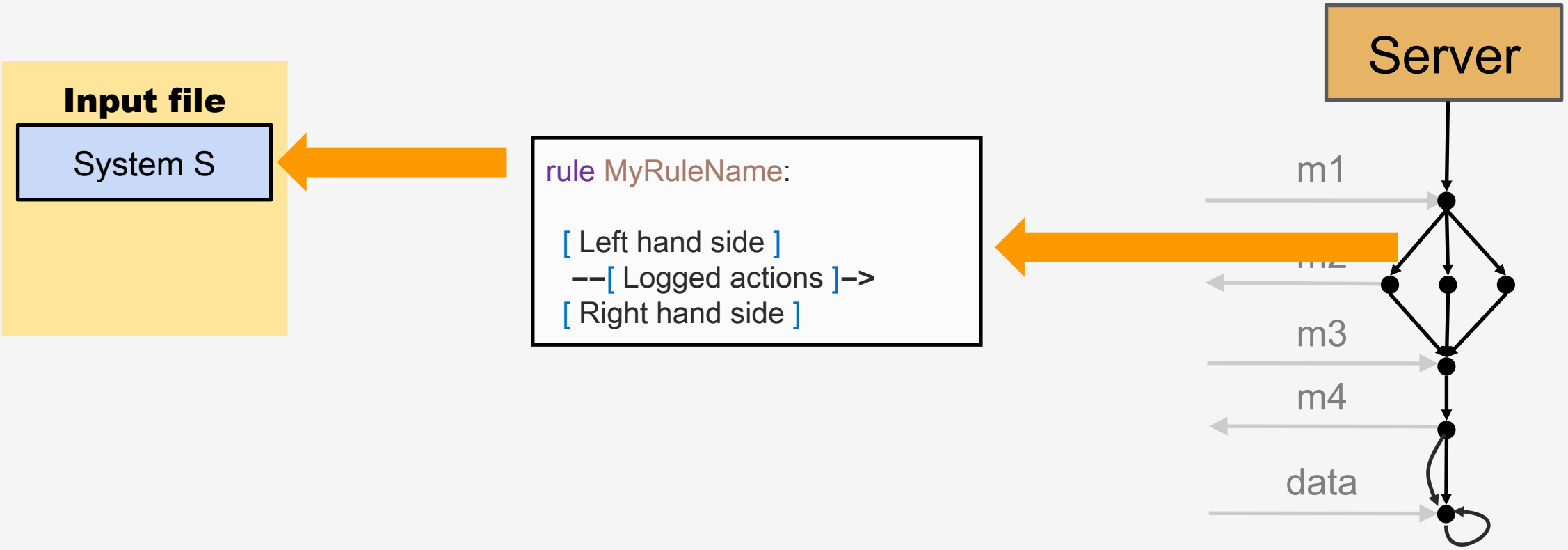
This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata,



Tamarin prover: workflow



Tamarin prover: workflow

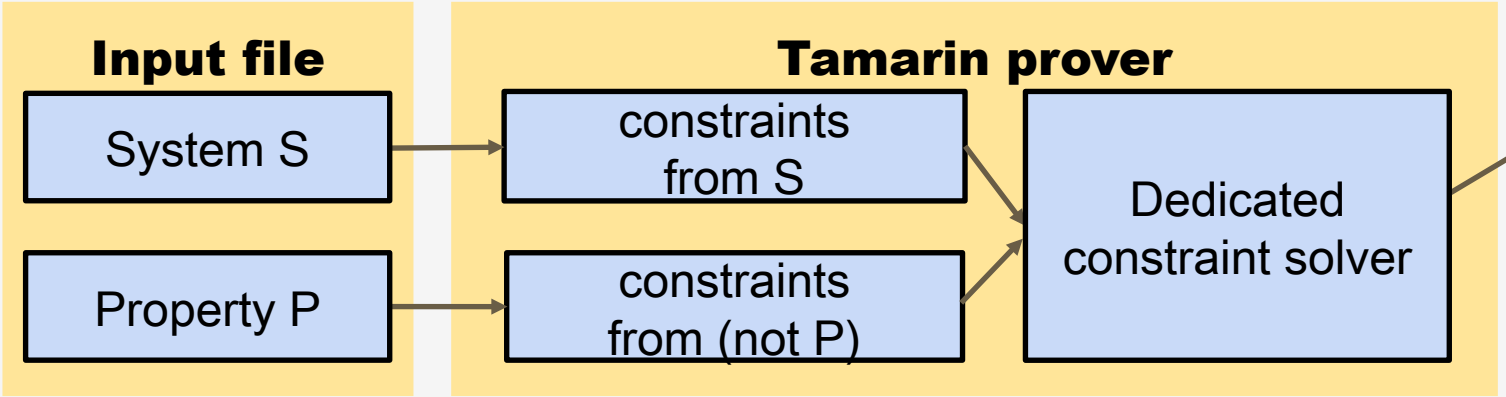
Input file

System S

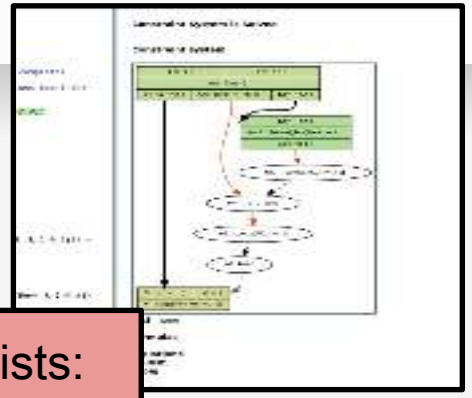
Property P

```
lemma Perfect_Forward_Secrecy:
  "All I R sessKey #i #k.
      SessionKey(I,R,sessKey) @ i &
      K(sessKey) @ k
      ==>
      ( (Ex #r. LtkReveal(I) @ r & r < i)
      | (Ex #r. LtkReveal(R) @ r & r < i)
      )
  "
```

Tamarin prover: workflow



Solution exists: ATTACK



Running Tamarin 1.9.0

Index Download Actions » Options »

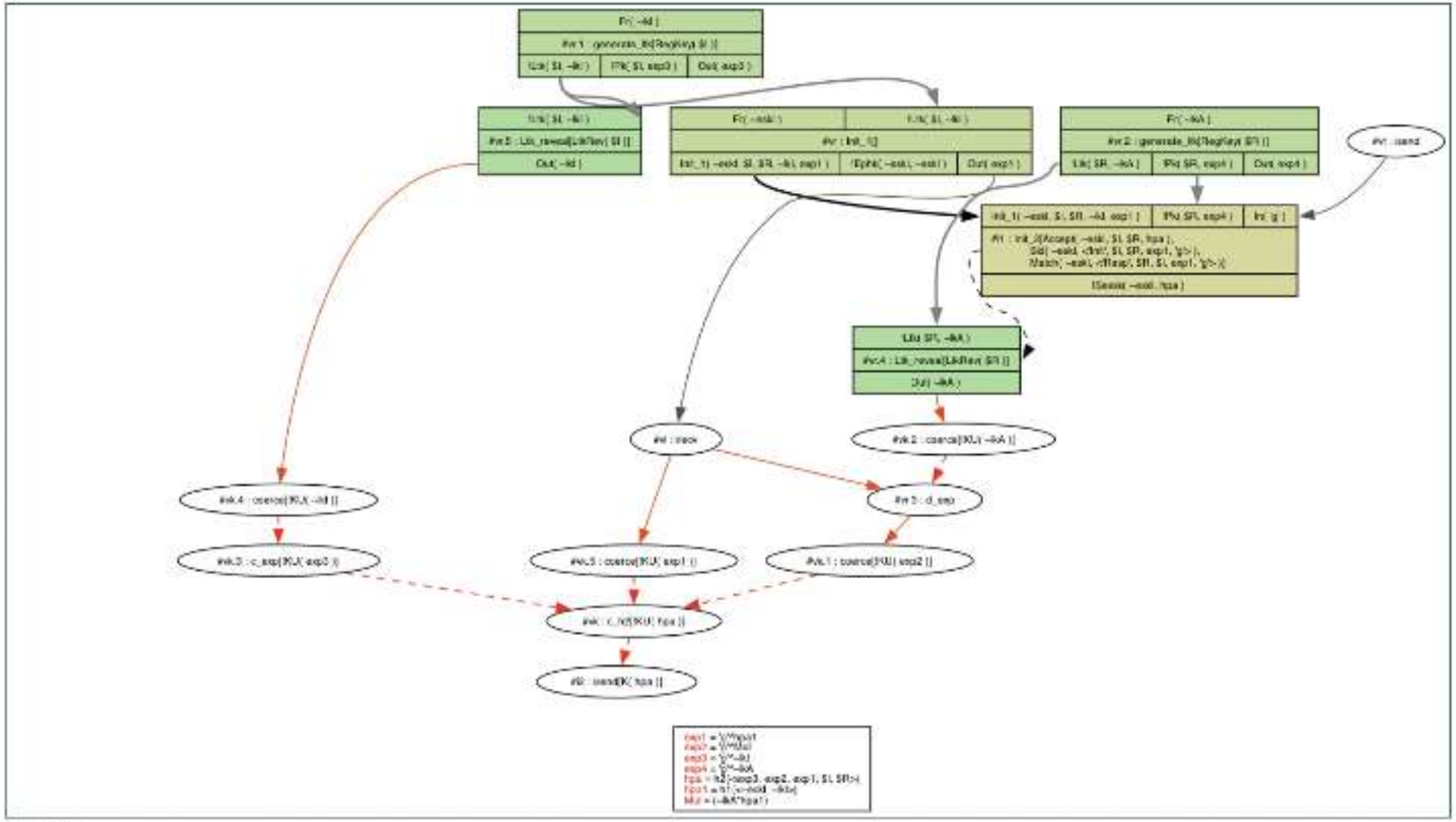
Proof scripts

```
Raw sources (11 cases, deconstructions complete)

Refined sources (11 cases, deconstructions complete)

lemma eCK_PFS_key_secrecy:
  all-traces
  "∀ #i1 #i2 Test A B k.
      ((Accept( Test, A, B, k ) @ #i1) ∧ (K( k ) @ #i2)
      ((((∃ #i3. SesskRev( Test ) @ #i3) ∨
         (∃ MatchingSession #i3 #i4 ms.
           ((Sid( MatchingSession, ms ) @ #i3) ∧ (Match(
           (∃ #i5. SesskRev( MatchingSession ) @ #i5)))
         (∃ MatchingSession #i3 #i4 ms.
           ((Sid( MatchingSession, ms ) @ #i5) ∧ (Match(
           ((∃ #i5 #i6. (LtkRev( A ) @ #i5) ∧ (EphkRev(
            (∃ #i5 #i6.
              (LtkRev( B ) @ #i5) ∧ (EphkRev( MatchingSe
        ((¬(∃ MatchingSession #i3 #i4 ms.
            (Sid( MatchingSession, ms ) @ #i3) ∧ (Match
         ((∃ #i5. (LtkRev( B ) @ #i5) ∧ (#i5 < #i1)) ∨
          (∃ #i5 #i6. (LtkRev( A ) @ #i5) ∧ (EphkRev( Te
simplify
solve( (∃ MatchingSession #i3 #i4 ms.
        (Sid( MatchingSession, ms ) @ #i3) ∧ (Match( Test
      ((∀ #i5. (LtkRev( B ) @ #i5) → ¬(#i5 < #i1)) ∧
       (∀ #i5 #i6. (LtkRev( A ) @ #i5) ∧ (EphkRev( Test )
  case case_2
  solve( Accept( Test, A, B, k ) @ #i1 )
    case Init_2
    solve( Init_1( ~eskI, $I, $R, ~lkI, hkI ) ▶₀ #i1 )
      case Init_1
      solve( !Pk( $R, pkR ) ▶₀ #i1 )
        case generate_ltk
        solve( !KU( h2(~m, 'g'^(~lkA*h1(<~eskI, ~lkI>)),
                       z.1, $I, $R>)
               ) @ #vk )
          case c_h2
          solve( !KU( 'g'^(~lkA*h1(<~eskI, ~lkI>)) ) @ #vk
            case Init_1
            solve( !KU( ~lkA ) @ #vk.11 )
              case Ltk_reveal
              solve( splitEqs(8) )
                case split_case_01
                solve( !KU( Y^~lkI ) @ #vk.6 )
                  case c_exp
                  solve( !KU( ~lkI ) @ #vk.12 )
                    case Ltk_reveal
                    solve( !KU( Y^h1(<~eskI, ~lkI>) ) @ #v
                      case Init_1
                      SOLVED // trace found
                    qed
                  qed
                qed
              qed
            qed
          qed
        qed
      qed
    qed
```

Visualization display

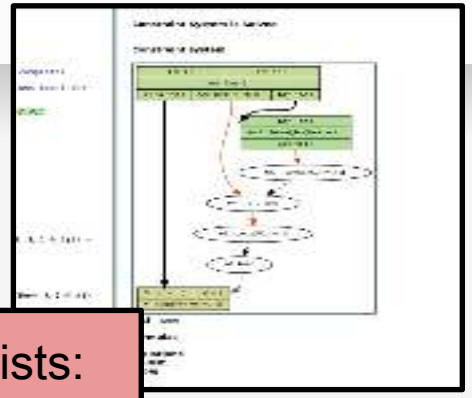
Constraint System is Solved

Constraint system



last: none

Tamarin prover: workflow



Input file

System S

Property P

Tamarin prover

constraints from S

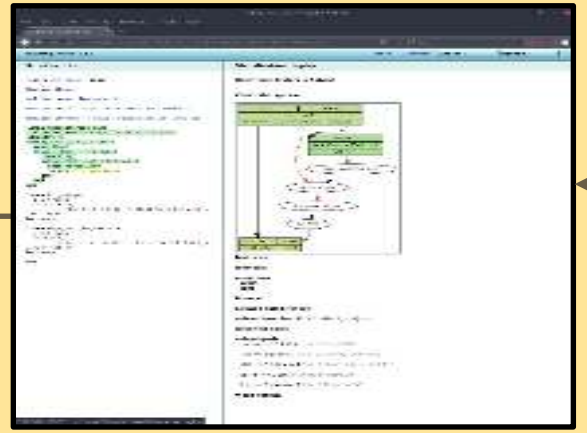
constraints from (not P)

Dedicated constraint solver

Solution exists: ATTACK

No solution exists: PROOF

Run out of time or memory



Interactive mode
(Inspect or guide partial proof)

Provide **hints** for the prover (e.g. invariants)

Innovation in techniques for deduction and symbolic reasoning

Tamarin's foundations

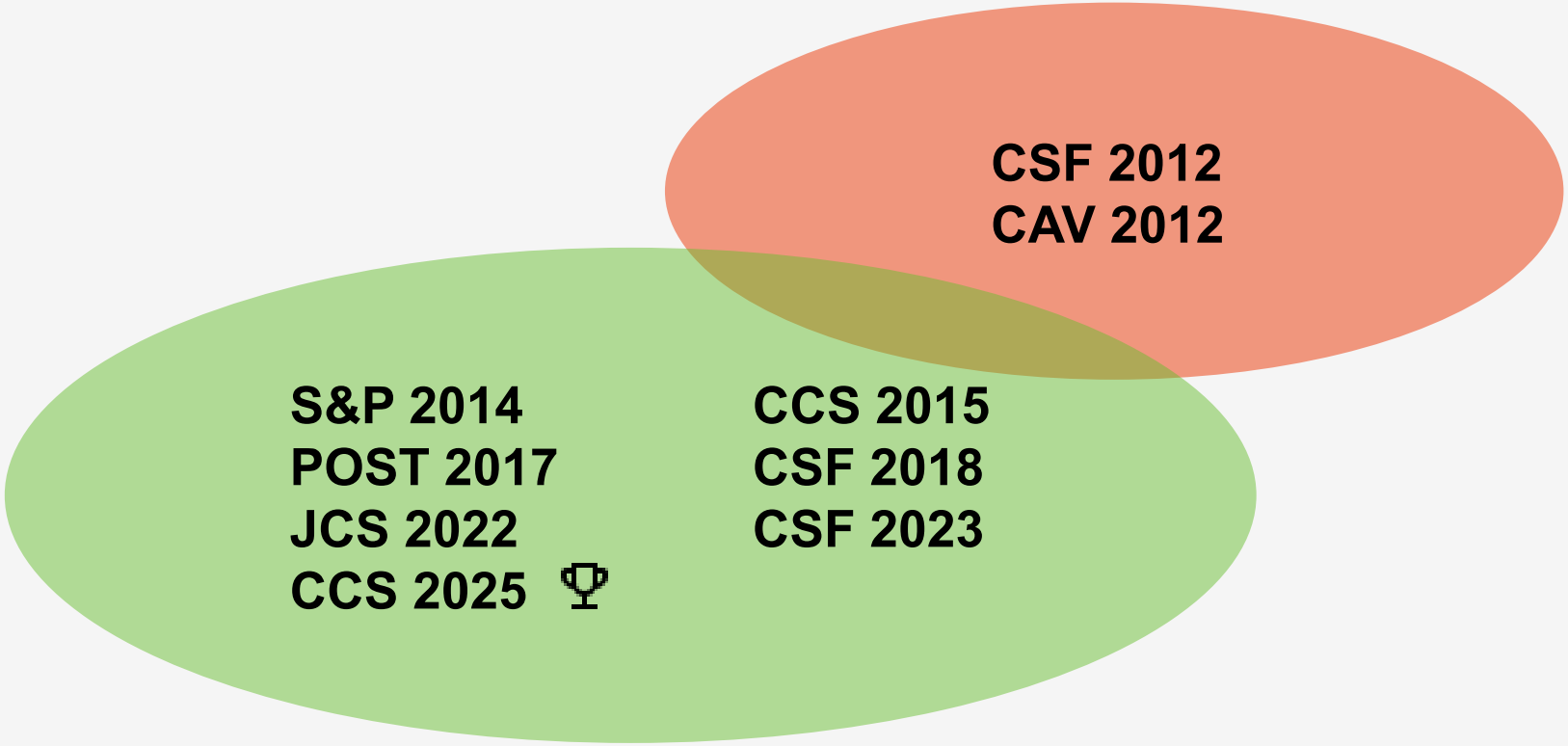
CSF 2012
CAV 2012

Operational semantics
multiset rewriting with equational theories

Property specification
fragment of first order logic with quantification over timepoints

Innovation in techniques for deduction and symbolic reasoning

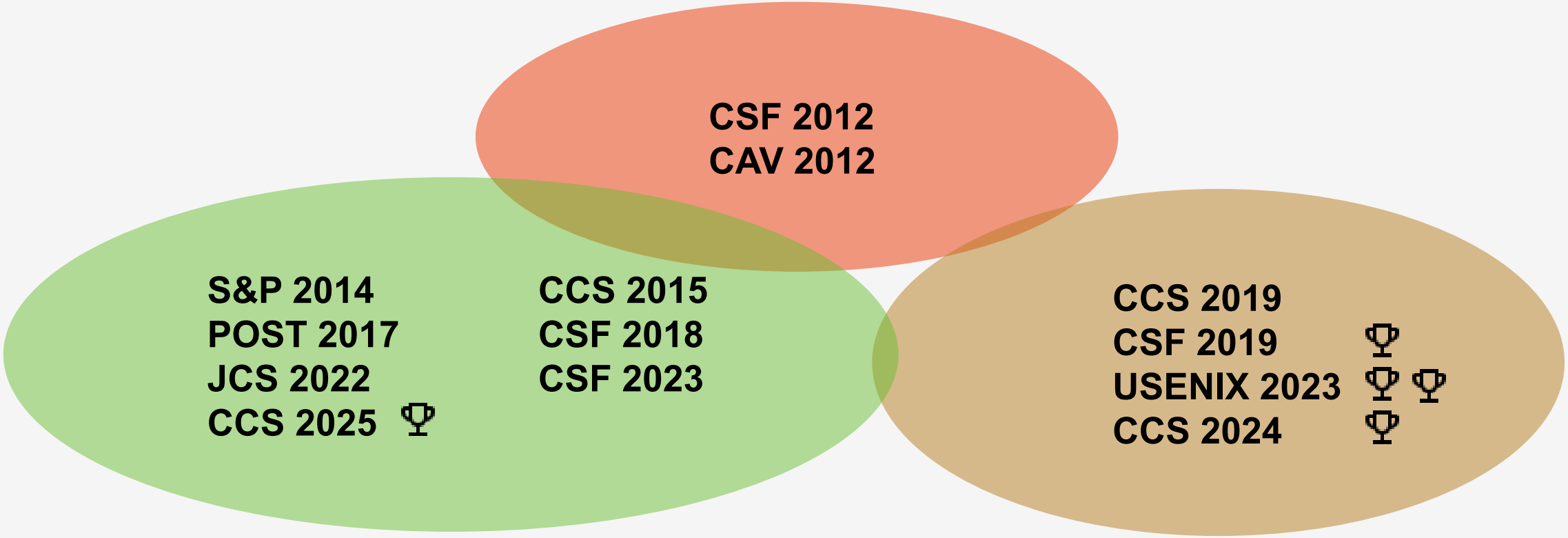
Tamarin's foundations



Algorithmic developments

Innovation in techniques for deduction and symbolic reasoning

Tamarin's foundations



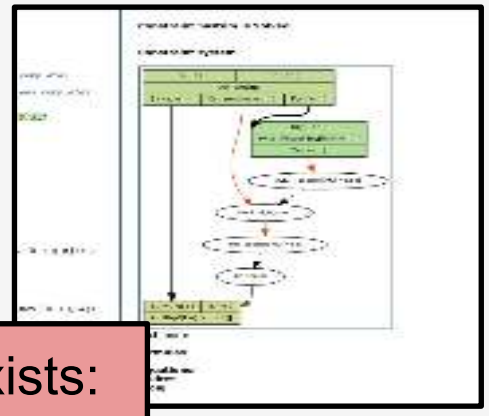
Algorithmic developments

More precise handling of cryptographic primitives

Innovation in techniques for deduction and symbolic reasoning

Dedicated constraint solving for undecidable problem, yet effective in practice

Algorithmic model checking for infinite-state systems



Solution exists: ATTACK

No solution exists: PROOF

Sound: if terminates with **falsified**, provides concrete counterexample/attack

Complete: if terminates with **verified**, then no counterexample / attack exists in model

Theory advancement and engineered for practical use

Abstraction levels in theory

Reality

Computational

Abstraction levels in theory

Reality

Computational

Symbolic

Abstraction levels in practice

Reality

Computational

Symbolic

State of the art: Attacks Tamarin can find

- Large attack scenarios (TLS 1.3 Rev 10: 18 messages)
- Cross-protocol attacks
- Unintended state machine transitions
- Downgrade attacks
- Nonce-reuse attacks (eg WiFi with AES-GCM)
- Invalid Curve Points
- Small order points
- DSKS attacks (Duplicate Signature Key Selection)
- Length extension attacks
- Maliciously generated keys
- Post-quantum secure protocols
- Hybrid schemes

These attack types are not hard-coded!

Tamarin finds these through systematic, accurate modeling of cryptography and protocols.

Success stories

[TLS] Revision 10: possible attack if client authentication is allowed during PSK

Sam Scott <sam.scott89@gmail.com> | Sat, 31 October 2015 11:19 UTC | Show header

Dear all,

We [1] are in the process of performing an automated symbolic analysis of the TLS 1.3 specification draft (revision 10) using the Tamarin prover [2],

While revis
authenticat ed
what we bel
authenticat
signature over the handshake hash, our Tamarin analysis finds an attack. The

TLS 1.3

EMV (Chip and Pin)

Security Research

Blog

iMessage PQ3

October 27, 2023

Advancing iMessage security: iMessage Contact Key Verification

CVE-2023-31127

SPDM (PCI Express)

OpenCVE > Vulnerabilities (CVE) > CVE-2023-31127

libspdm is a sample implementation that follows the DMTF SPDM specificat
in libspdm prior to version 2.3.1. If a device supports both DHE session and

CVSS v3 8.8 HIGH

8.8 /10

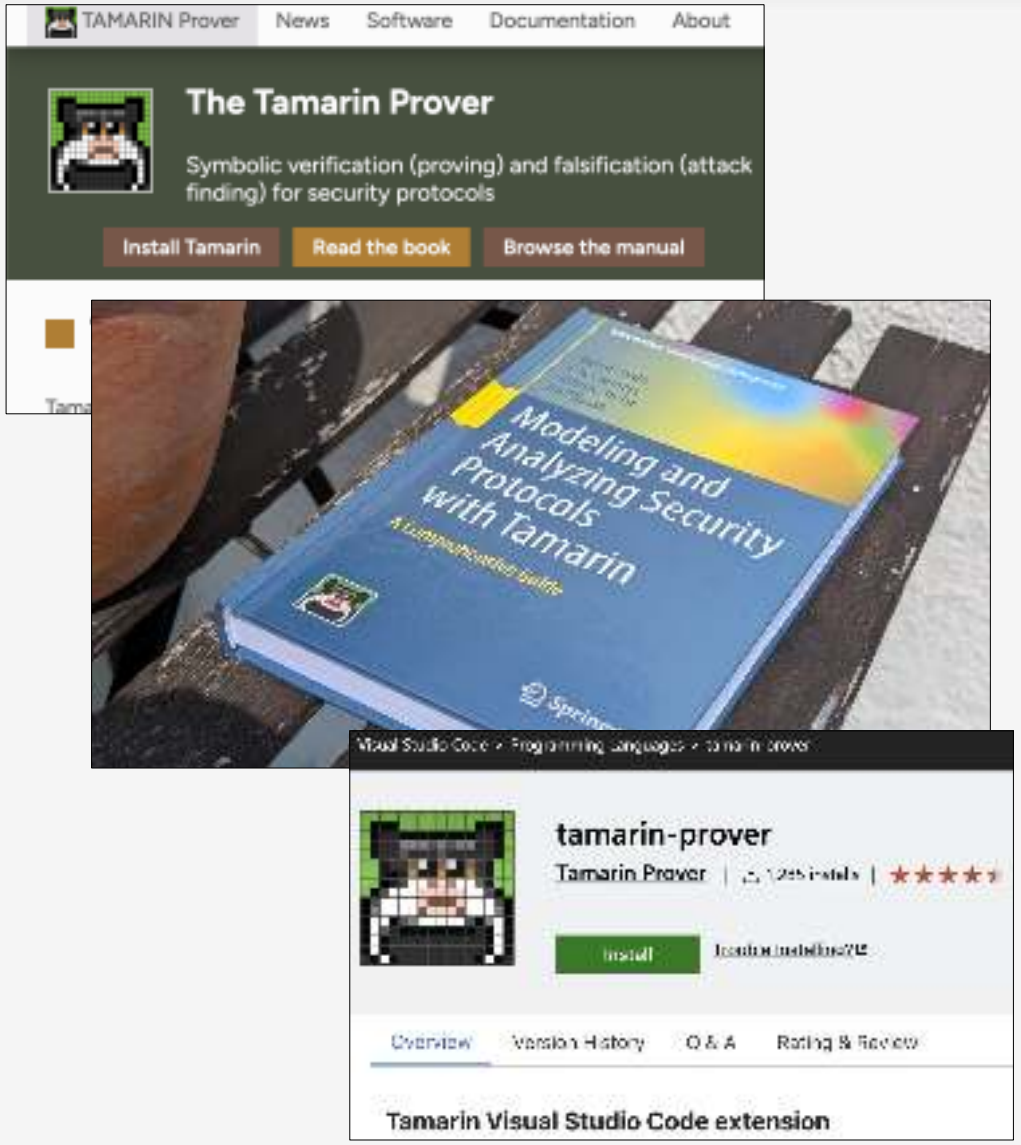
Tamarin use in industry

Maturity and development

Tamarin is a mature tool: you can use it now
`tamarin-prover.com`

- Open source development on github
- Active user community
- Teaching materials including book (free PDF)
- Many example protocol models
- Plugins for editors (e.g., VSC)

Under active development



Formal security analysis and the future

More memory-safe languages

Better standardized cryptography

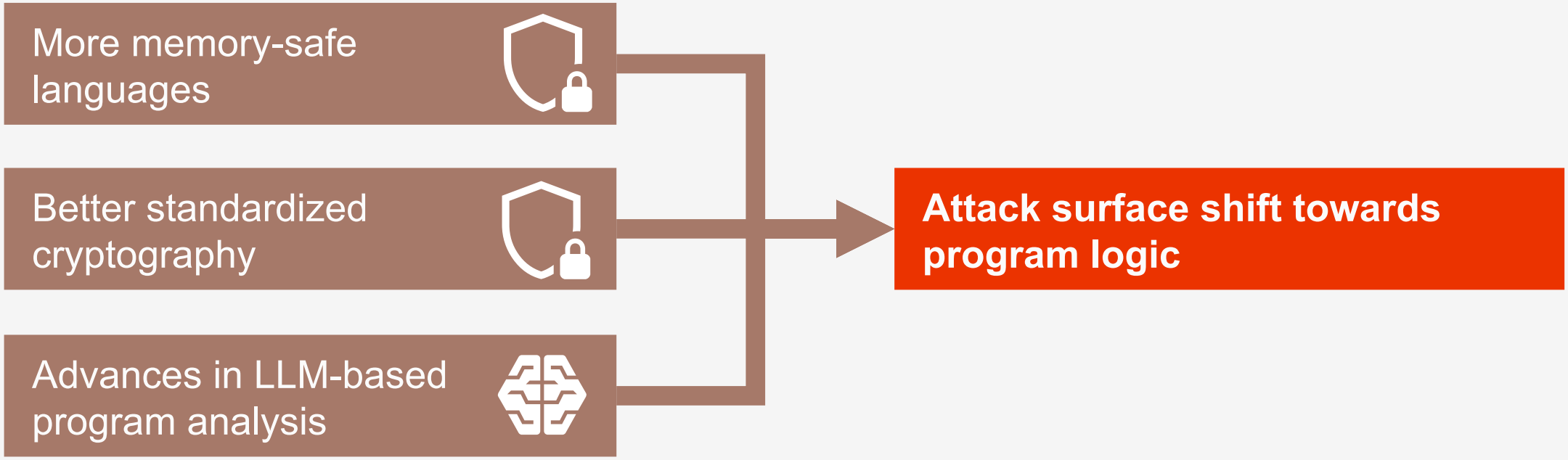
Formal security analysis and the future

More memory-safe languages

Better standardized cryptography

Advances in LLM-based program analysis

Formal security analysis and the future



Formal security analysis and the future

More memory-safe languages

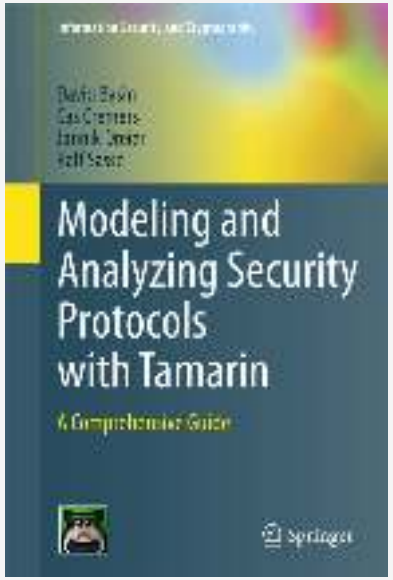
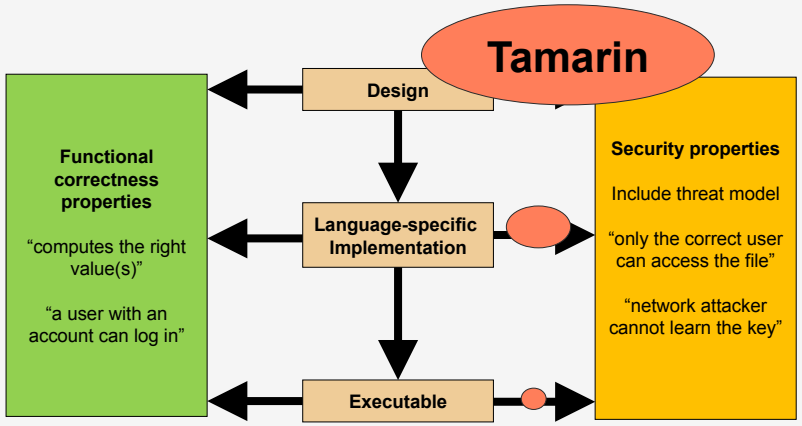
Better standardized cryptography

Advances in LLM-based program analysis

Attack surface shift towards program logic

Formal analysis tools such as Tamarin crucial for hardening (or finding attacks)

Conclusions

- tamarin-prover.com
- Open source, well documented, and ready for use
- In preparation: Tamarin foundation
- Reach out to us or the Tamarin mailing list

Cas Cremers (cremers@cispa.de)